CODE DIGEST
All your tech in one place

sfi CET

ACADEMIC SUPPORT CELL
An Initiative by SFI CET Unit

85 ESSENTIAL DBMS INTERVIEW QUESTIONS

YOU SHOULD KNOW

Swipe To Learn

85 ESSENTIAL DBMS INTERVIEW QUESTIONS

01 WHAT IS A DATABASE MANAGEMENT SYSTEM (DBMS) ?

a dbms is software that acts as an interface between users and the database, allowing users to create, store, retrieve, and manage data efficiently while ensuring security and consistency.

02 FILE SYSTEM VS. DBMS: WHY IS DBMS BETTER?

DBMS minimizes data redundancy, ensures consistency, supports safe concurrent access, and strictly enforces integrity constraints and security —all of which traditional file systems lack.

03 WHAT IS DATA INDEPENDENCE?

The ability to modify the schema at one level of the database system without having to alter the schema or applications at the next higher level.

04 WHAT IS THE INTERNAL LEVEL (PHYSICAL SCHEMA)?

The lowest level of data abstraction that describes exactly how the data is physically stored on the disk (file structures, indexing, and compression).

05 WHAT IS THE CONCEPTUAL LEVEL (LOGICAL SCHEMA)?

The middle level that describes what data is stored in the entire database and the relationships among that data, completely hiding the physical storage details.

06 WHAT IS THE EXTERNAL LEVEL (VIEW SCHEMA)?

The highest level that describes only the specific part of the database relevant to a particular user, allowing customized views of the same data.

07 WHAT IS DATA DEFINITION LANGUAGE (DDL) ?

SQL commands used to define or modify the database structure. Includes CREATE, ALTER, DROP, and TRUNCATE.

08 WHAT IS DATA MANIPULATION LANGUAGE (DML) ?

SQL commands used to manipulate the actual data stored within the tables. Includes SELECT, INSERT, UPDATE, and DELETE.

09 WHAT IS DATA CONTROL LANGUAGE (DCL) ?

SQL commands used to manage permissions and access controls. Includes GRANT and REVOKE.

10 WHAT IS TRANSACTION CONTROL LANGUAGE (TCL) ?

SQL commands used to manage transactions to maintain database consistency. Includes COMMIT, ROLLBACK, and SAVEPOINT.

11 WHAT IS QUERY PROCESSING?

The sequence of steps a DBMS takes to execute an SQL statement: Parsing (syntax check) → Translation (to relational algebra) → Optimization (finding the best execution plan) → Execution.

12 WHAT IS AN ENTITY?

A real-world object with an independent existence (e.g., a specific student).

13 WHAT IS AN ENTITY SET?

A collection of entities of the exact same type (e.g., all students in a school).

14 WHAT IS A WEAK ENTITY?

An entity that cannot be uniquely identified by its own attributes alone. It relies on a relationship with a Strong Entity and uses a partial key.

15 WHAT ARE THE DIFFERENT TYPES OF ATTRIBUTES?

- Simple: Indivisible (e.g., Age).
- Composite: Can be divided into sub-parts (e.g., Name = First + Last).
- Multi-valued: Can have multiple values (e.g., Phone Numbers).
- Derived: Value is calculated dynamically (e.g., Age derived from DOB .

16 **WHAT IS CARDINALITY RATIO?**

Defines the maximum number of relationship instances an entity can participate in: One-to-One (1:1), One-to-Many (1:N), Many-to-One (N:1), and Many-to-Many (M:N).

17 **WHAT ARE PARTICIPATION CONSTRAINTS?**

- Total Participation: Every entity in a set must participate in the relationship.
- Partial Participation: Not all entities are required to participate.

18 **WHAT IS A CANDIDATE KEY?**

A minimal set of attributes that can uniquely identify a row in a table.

19 **WHAT IS A PRIMARY KEY?**

The specific Candidate Key chosen to uniquely identify rows. It cannot contain NULL values.

20 **WHAT IS A SUPER KEY?**

Any combination of attributes that uniquely identifies a row. All Candidate Keys are Super Keys, but not all Super Keys are Candidate Keys).

21 **WHAT IS A FOREIGN KEY?**

An attribute in one table that refers to the Primary Key of another table, enforcing relationship links.

22 **WHAT ARE PRIME VS. NON-PRIME ATTRIBUTES?**

- Prime Attribute: An attribute that is a part of any candidate key.
- Non-Prime Attribute: An attribute that is not a part of any candidate key.

23 **WHAT IS AN INTEGRITY CONSTRAINT?**

A set of rules applied to a database schema to guarantee that the data remains accurate, valid, and consistent during operations.

24 **WHAT ARE THE TYPES OF INTEGRITY CONSTRAINTS?**

Domain Integrity, Entity Integrity, and Referential Integrity.

---

25 **WHAT IS DOMAIN INTEGRITY?**

Ensures that data entered into a column strictly meets the defined data type, format, and range for that column.

---

26 **WHAT IS ENTITY INTEGRITY?**

Ensures that every table has a Primary Key, and that this Primary Key is both unique and not NULL.

---

27 **WHAT IS REFERENTIAL INTEGRITY?**

Ensures that a Foreign Key value must either match an existing Primary Key value in the referenced table, or be NULL.

---

28 **WHAT IS THE SELECT (σ) OPERATION?**

Filters and retrieves entire rows from a relation that satisfy a specific condition.

---

29 **WHAT IS THE PROJECT (Π) OPERATION?**

Selects specific columns from a relation and discards the rest, removing duplicate rows.

---

30 **WHAT IS THE UNION (∪) OPERATION?**

Combines rows of two relations, returning all unique rows from both.

---

31 **WHAT IS THE CARTESIAN PRODUCT (X) OPERATION?**

Combines every single row of the first table with every single row of the second table.

---

32 **WHAT IS A NATURAL JOIN (⋈) ?**

Automatically joins two tables based on all attributes they share in common, removing the duplicate matching columns from the result.

---

33 WHAT IS A FUNCTIONAL DEPENDENCY (FD) ?

A rule stating that if two rows have the same value for attribute X, they must have the same value for attribute Y (X → Y).

34 WHAT ARE DATA ANOMALIES?

Flaws in bad database design:

- Insertion Anomaly: Cannot insert data because other data is missing.
- Deletion Anomaly: Deleting a row unintentionally deletes unrelated vital data.
- Update Anomaly: Updating a value requires changing multiple rows.

35 WHAT IS FIRST NORMAL FORM (1NF)?

A table where every cell contains a single, atomic value. No arrays or repeating groups are allowed.

36 WHAT IS SECOND NORMAL FORM (2NF) ?

A table that is in 1NF and contains absolutely no Partial Dependencies (where an attribute depends on only part of a composite primary key).

37 WHAT IS THIRD NORMAL FORM (3NF) ?

A relation is in 3NF if, for every non-trivial functional dependency X → A, either X is a Super Key OR A is a Prime Attribute.

38 WHAT IS BOYCE-CODD NORMAL FORM (BCNF) ?

A stricter version of 3NF where, for every non-trivial functional dependency X → Y, X must be a Super Key.

39 WHAT IS LOSSLESS VS. LOSSY DECOMPOSITION?

- Lossless: Joining decomposed tables back together yields the exact original table without fake rows.
- Lossy: Joining decomposed tables generates extra, incorrect rows that were not in the original.

40 EXPLAIN SQL CONSTRAINTS.

Keywords that enforce rules on data: NOT NULL (prevents empty values), UNIQUE (prevents duplicates), CHECK (must meet a condition), and DEFAULT (fallback value).

41 WHAT ARE REFERENTIAL ACTIONS?

Rules defined on Foreign Keys for when parent data is updated/deleted: CASCADE (auto-delete/update child), SET NULL (child becomes NULL , and RESTRICT (prevents parent deletion).

42 WHAT IS NULL IN SQL?

NULL represents missing, unknown, or inapplicable data. It is not equivalent to zero or a blank space.

43 GROUP BY VS. HAVING VS. WHERE?

- WHERE: Filters rows before aggregation.
- GROUP BY: Groups identical rows into summary rows.
- HAVING: Filters the summary groups after aggregation.

44 WHAT ARE AGGREGATE FUNCTIONS?

Functions that operate on a set of rows to return a single calculated result (e.g., SUM, COUNT, MIN, MAX, AVG).

45 DELETE VS. TRUNCATE VS. DROP?

- DELETE: Removes specific rows, logs the deletion, can be rolled back.
- TRUNCATE: Empties the entire table instantly, does not log rows, usually cannot be rolled back.
- DROP: Completely destroys the table structure and its data.

46 WHAT IS AN INNER JOIN?

Returns only the records that have matching values in both tables.

47 WHAT IS A LEFT OUTER JOIN?

Returns all records from the left table, and the matched records from the right table (fills with NULLs if no match).

48 **WHAT IS A RIGHT OUTER JOIN?**

Returns all records from the right table, and the matched records from the left table (fills with NULLs if no match).

---

49 **WHAT IS A FULL OUTER JOIN?**

Returns all records when there is a match in either the left or the right table.

---

50 **WHAT IS A CROSS JOIN?**

Returns the Cartesian product (all possible combinations of rows from both tables).

---

51 **WHAT IS A SELF JOIN?**

Joins a table with itself to compare rows within the same table.

---

52 **UNION VS. UNION ALL?**

Both combine result sets. UNION removes duplicates; UNION ALL includes duplicates and is much faster.

---

53 **WHAT ARE SUBQUERIES?**

A nested query inside another query. A Correlated Subquery executes repeatedly, once for every row evaluated by the outer query.

---

54 **WHAT ARE SQL WINDOW FUNCTIONS?**

Functions that perform calculations across a set of table rows that are related to the current row, without collapsing them into a single output row (unlike aggregate functions).

---

55 **WHAT IS THE DIFFERENCE BETWEEN RANK() , DENSE_RANK() , AND ROW_NUMBER() ?**

- ROW_NUMBER(): Assigns a unique, sequential integer (e.g., 1, 2, 3, 4 .
- RANK(): Assigns the same rank to duplicates, but skips subsequent numbers (e.g., 1, 1, 3 .
- DENSE_RANK(): Assigns the same rank to duplicates and does not skip numbers (e.g., 1, 1, 2 .

56 **WHAT IS A VIEW?**

A virtual table that dynamically pulls data from base tables to simplify queries or restrict access.

57 **WHAT IS A TRIGGER?**

A stored script that automatically executes in response to INSERT, UPDATE, or DELETE events on a table.

58 **WHAT IS A STORED PROCEDURE?**

A precompiled, reusable collection of SQL statements containing parameters and logic.

59 **WHAT IS A TRANSACTION?**

A logical unit of work consisting of one or more SQL statements that must execute completely or not at all.

60 **WHAT ARE THE ACID PROPERTIES?**

- Atomicity: "All or nothing" execution.
- Consistency: Moves the DB from one valid state to another.
- Isolation: Transactions run unaware of each other's intermediate states.
- Durability: Committed changes survive system crashes.

61 **WHAT IS THE HIERARCHY OF SCHEDULES?**

A strictness hierarchy for database safety: Recoverable → Cascadeless → Strict.

62 **WHAT IS A RECOVERABLE SCHEDULE?**

A schedule where a transaction commits only after all transactions whose changes it has read have successfully committed.

63 **WHAT IS A CASCADELESS SCHEDULE?**

A schedule where transactions never read uncommitted data, preventing a single failure from causing cascading rollbacks of other transactions.

64 **WHAT IS A STRICT SCHEDULE?**

A schedule where a transaction cannot read or write a data item until the transaction that last wrote it has committed or aborted.

65 **WHAT IS 2-PHASE LOCKING (2PL) ?**

A protocol guaranteeing serializability. It requires a Growing Phase (acquire locks, release none) and a Shrinking Phase (release locks, acquire none).

66 **WHAT IS STRICT 2-PHASE LOCKING (STRICT 2PL) ?**

A stricter variation of 2PL where all exclusive (write) locks must be held until the transaction fully commits or aborts, ensuring a Strict Schedule.

67 **WHAT IS A DEADLOCK AND HOW IS IT HANDLED?**

When transactions wait indefinitely for locks held by each other. Prevented using timestamp protocols Wait-Die, Wound-Wait) and detected by checking Wait-For Graphs for cycles.

68 **WHAT IS LOG-BASED RECOVERY?**

A recovery method that records all database modifications in a secure log file before applying them, allowing the system to Undo or Redo transactions after a crash.

69 **WHAT IS WRITE-AHEAD LOGGING (WAL) ?**

A strict rule stating that a log record representing a modification must be physically written to disk before the actual data page is updated.

70 **WHAT IS DEFERRED UPDATE?**

Changes are recorded only in the log; actual database updates are delayed until the transaction commits. Primarily requires REDO on crash.

71 **WHAT IS IMMEDIATE UPDATE?**

Database pages may be updated immediately before commit. Requires both UNDO (for uncommitted) and REDO (for committed) upon crash

72 WHAT IS A CHECKPOINT?

A recovery mechanism that periodically flushes modified data to disk and records a safe point in the log, meaning the DBMS doesn't need to process the entire log during crash recovery.

73 WHAT IS SHADOW PAGING?

Maintains two page tables (current and shadow). When a transaction commits, the shadow table replaces the current one, allowing recovery without log-based UNDO.

74 WHAT IS INDEXING AND WHAT IS ITS PRIMARY TRADEOFF?

Indexing uses data structures to quickly locate records. Tradeoff: Drastically speeds up read queries SELECT , but slows down write operations INSERT, UPDATE, DELETE because the index must be constantly updated.

75 CLUSTERED VS. NON-CLUSTERED INDEX?

- Clustered Index: Determines and strictly follows the physical ordering of data rows on the disk.
- Non-Clustered Index: A separate index structure containing pointers to the records. Data is not physically ordered by this index.

76 WHAT IS A MULTILEVEL INDEX?

An index built over another index. Used to reduce disk accesses when the primary index becomes too large to fit in memory.

77 WHAT IS A B- TREE?

A self-balancing search tree where data pointers and search keys are stored in both internal (non-leaf) nodes and leaf nodes.

78 WHAT IS A B+ TREE?

A search tree where data pointers are stored only in leaf nodes (internal nodes are just routing keys), and leaf nodes are linked together in a chain.

79 **B- TREE VS. B+ TREE?**

Because B Trees link their leaf nodes and keep all data at the bottom level, they are vastly superior for range queries compared to standard B Trees.

80 **WHAT IS STATIC HASHING?**

Hashing with a fixed number of data buckets. Prone to performance drops and overflow chains as the database grows.

81 **WHAT IS DYNAMIC (EXTENDIBLE) HASHING?**

Hashing where the number of buckets grows or shrinks dynamically using a directory of pointers, preventing overflow issues.

82 **WHAT IS THE CAP THEOREM?**

A rule stating that distributed databases can only guarantee two out of three characteristics simultaneously: Consistency, Availability, and Partition Tolerance.

83 **WHAT IS CONSISTENCY (IN CAP) ?**

Every read receives the most recent write or an error. (All nodes see the exact same data at the same time).

84 **WHAT IS AVAILABILITY (IN CAP ?**

Every request receives a non-error response, without the guarantee that it contains the most recent write.

85 **WHAT IS PARTITION TOLERANCE (IN CAP) ?**

The system continues to operate and process requests despite network drops or communication breaks between nodes.